Relatório Técnico de Entrega 6 — Comunicação Bidirecional e Confirmação de Comandos

Curso: Internet das Coisas III — IFMG Campus Ouro Branco

Professor: Charles Tim Batista Garrocho

Aluno: Daniel

Dispositivo: `esp32_temp` (ESP32 DevKit V1)

Projeto: Sistema de Alerta de Temperatura e Saúde Ambiental IoT

Data: Outubro / 2026

---

1. Resumo Executivo da Entrega

A **Entrega 6** conclui o ciclo da **comunicação bidirecional e confirmação de comandos** entre o Gateway Node.js e o dispositivo embarcado ESP32 através do protocolo MQTT.

O objetivo fundamental desta entrega é garantir que a aplicação não apenas emita comandos para o dispositivo, mas que aguarde e processe uma confirmação formal de execução (*acknowledgement* ou mensagem de estado retido).

Com isso, o sistema passa a **distinguir com precisão entre um comando apenas emitido pelo gateway e um comando efetivamente executado pelo atuador físico no ESP32**, atualizando o banco de dados chave-valor Redis única e exclusivamente após a confirmação bem-sucedida do nó IoT.

---

2. Tabela de Conformidade dos Produtos Esperados

#	Produto Esperado	Requisito da Disciplina	Status	Implementação / Evidência
1	**Publicação de comandos via MQTT**	Gateway emite comando no tópico apropriado	**100% Atendido**	`backend/src/mqtt/gateway_mqtt.ts:170-208`
2	**Recebimento de comandos no ESP32**	ESP32 assina e faz o parsing de comando JSON	**100% Atendido**	`firmware/src/main.cpp:14-63`
3	**Acionamento remoto do atuador**	Hardware aciona o LED (GPIO 12) e Relé (GPIO 14)	**100% Atendido**	`firmware/src/actuator_manager.cpp:13-17`
4	**Publicação da confirmação pelo ESP32**	Dispositivo publica estado no tópico `.../state`	**100% Atendido**	`firmware/src/mqtt_client.cpp:73-98`
5	**Recepção da confirmação pelo Gateway**	Gateway intercepta o retorno com validação Zod	**100% Atendido**	`backend/src/mqtt/gateway_mqtt.ts:153-167`
6	**Atualização do atuador no Redis**	Estado no Redis é atualizado com o valor confirmado	**100% Atendido**	`backend/src/storage/kv_store.ts`
7	**Evidência do fluxo bidirecional**	Demonstração ponta a ponta com tempos e logs	**100% Atendido**	Requisição REST e logs reais do terminal
8	**Garantia de integridade do estado**	Estado no banco só altera após confirmação real	**100% Atendido**	Teste de timeout 504 em nó offline

3. Diagrama e Arquitetura do Fluxo Bidirecional

■ Log de Execução / Diagnóstico

```
[Cliente Web / REST API]
       ■  1. POST /api/devices/esp32_temp/command
       ▼
[Gateway Node.js]
       ■  2. Gera requestId ("cmd_...") e publica comando
       ■     Tópico: ifmg/iot3/turmaA/daniel/esp32_temp/command
       ▼
[Broker Mosquitto]
       ■  3. Roteia comando para o cliente inscrito
       ▼
[ESP32 DevKit V1]
       ■  4. handleMqttCommand() processa JSON
       ■  5. ActuatorManager aciona GPIO 12 e GPIO 14
       ■  6. Publica confirmação com retain=true
       ■     Tópico: ifmg/iot3/turmaA/daniel/esp32_temp/state
       ▼
[Broker Mosquitto]
       ■  7. Encaminha confirmação ao Gateway
       ▼
[Gateway Node.js]
       ■  8. Intercepta ACK no sub-tópico "state"
       ■  9. Atualiza chave "device:esp32_temp:presence" no Redis
       ■ 10. Resolve Promise pendente e responde HTTP 200 OK
       ▼
[Cliente Web / REST API] (Exibe confirmação confirmada pelo hardware)
```

4. Trechos de Código-Fonte Relevantes

4.1 Envio de Comando com Registro de Confirmação Pendente (`backend/src/mqtt/gateway_mqtt.ts`)

■ TypeScript / Gateway Node.js

```
async sendCommand(
  deviceId: string,
  action: string,
  state: boolean,
  timeoutMs = 5000
): Promise<any> {
  if (!this.client || !this.isConnected) {
    throw new Error('MQTT_DISCONNECTED: Gateway desconectado.');
  }

  const commandId = `cmd_${Math.random().toString(36).substring(2, 9)}`;
  const topicIfmg = `ifmg/iot3/turmaA/daniel/${deviceId}/command`;

  const payload = JSON.stringify({
    action: 'set',
    target: 'led',
    value: state,
    requestId: commandId,
    commandId,
    state,
  });

  return new Promise((resolve, reject) => {
    // Timeout estrito de 5 segundos se o dispositivo nao responder
    const timeoutTimer = setTimeout(() => {
      this.pendingCommands.delete(commandId);
      reject(new Error(
        'ACTUATOR_ACK_TIMEOUT: Dispositivo nao respondeu ao ACK em 5s.'
      ));
    }, timeoutMs);

    // Registra comando pendente aguardando confirmacao via MQTT
    this.pendingCommands.set(commandId, { resolve, reject, timeoutTimer
        });

    this.client?.publish(topicIfmg, payload, { qos: 1 }, (err) => {
      if (err) {
        clearTimeout(timeoutTimer);
        this.pendingCommands.delete(commandId);
        reject(err);
      }
    });
  });
}
```

4.2 Recepção do Comando e Acionamento Físico no ESP32 (`firmware/src/main.cpp`)

■ C++ / Firmware ESP32

```
void handleMqttCommand(char* topic, byte* payload, unsigned int length)
    {
    StaticJsonDocument<384> doc;
    DeserializationError error = deserializeJson(doc, payload, length);
    if (error) return;

    const char* action = doc["action"] | "";
    const char* target = doc["target"] | "led";
    const char* requestId = doc["requestId"] | (doc["commandId"] |
        "req-default");

    bool hasState = doc.containsKey("value") ||
        doc.containsKey("state");
    bool requestedState = doc.containsKey("value")
        ? doc["value"].as<bool>()
        : doc["state"].as<bool>();

    if (String(action).equalsIgnoreCase("set") || String(action) ==
        "SET_ACTUATOR") {
        if (!hasState) return;

        // 1. Executa o acionamento fisico no hardware
        ActuatorManager::setActuatorState(requestedState);

        // 2. Publica imediatamente a confirmacao formal de estado com
            retain=true
        MQTTClientManager::publishState(target, requestedState,
            requestId);
        Serial.println("[MQTT] Confirmacao publicada no topico
            .../state.");
    }
}
```

4.3 Publicação da Confirmação de Estado pelo ESP32 (`firmware/src/mqtt_client.cpp`)

■ C++ / Firmware ESP32

```
bool MQTTClientManager::publishState(
    const char* actuator,
    bool state,
    const char* requestId
) {
    if (!client.connected()) return false;

    StaticJsonDocument<256> doc;
    doc["deviceId"] = DEVICE_ID;
    doc["actuator"] = (actuator && strlen(actuator) > 0) ? actuator :
        "led";
    doc["state"] = state;
    if (requestId && strlen(requestId) > 0) {
        doc["requestId"] = requestId;
    }
    doc["timestamp"] = millis() / 1000;

    char buffer[256];
    serializeJson(doc, buffer);

    // Publica no topico .../state com flag retained = true e QoS 1
    return client.publish(TOPIC_STATE, buffer, true);
}
```

4.4 Recepção do ACK no Gateway e Atualização no Redis (`backend/src/mqtt/gateway_mqtt.ts`)

■ TypeScript / Gateway Node.js

```
} else if (subtopic === 'commands/ack' || subtopic === 'state') {
  const rawJson = JSON.parse(messageStr);
  const parsed = ackSchema.parse(rawJson);

  console.log(
    `[MQTT Confirmação/State] [${deviceId}] ` +
    `Req: ${parsed.commandId} | Estado: ${parsed.state}`
  );

  // Atualiza o estado confirmado do atuador no Redis
  const record = await kvStore.getDeviceRecord(deviceId);
  if (record && record.state) {
    record.state.actuatorState = parsed.state;
    await kvStore.saveTelemetry(deviceId, record.state);
  }

  // Libera a requisicao HTTP com sucesso
  const pending = this.pendingCommands.get(parsed.commandId);
  if (pending) {
    clearTimeout(pending.timeoutTimer);
    this.pendingCommands.delete(parsed.commandId);
    pending.resolve(parsed);
  }
}
```

5. Evidências de Funcionamento e Execução

5.1 Envio de Comando e Confirmação Imediata via cURL REST

■ Terminal / Comandos de Execução

```
$ curl -s -X POST -u admin:admin123 \
  -H "Content-Type: application/json" \
  -d '{"action":"set","target":"led","state":true}' \
  "http://localhost:5001/api/devices/esp32_temp/command"

{
  "success": true,
  "commandId": "cmd_p3iyyza",
  "deviceId": "esp32_temp",
  "actuatorState": true,
  "acknowledgedAt": "2026-10-15T23:27:18.837Z"
}
```

5.2 Verificação de Atualização Efetiva no Banco de Dados Redis

■ Terminal / Comandos de Execução

```
$ docker exec iot-redis redis-cli get device:esp32_temp:presence
{
  "deviceId": "esp32_temp",
  "status": "online",
  "lastSeen": "2026-10-15T23:33:51.025Z",
  "state": {
    "deviceId": "esp32_temp",
    "sensor": "dht22_temperatura",
    "value": 23.7,
    "unit": "C",
    "actuatorState": true,
    "recordedAt": "2026-10-15T23:33:51.025Z"
  }
}
```

5.3 Evidência de Rejeição e Proteção em Dispositivo Offline (Timeout 504)

Ao enviar o comando para um nó desconectado (`esp32_inexistente`), o Gateway protege a persistência contra comandos não executados:

■ Terminal / Comandos de Execução

```
$ curl -s -X POST -u admin:admin123 \
  -H "Content-Type: application/json" \
  -d '{"action":"set","target":"led","state":true}' \
  "http://localhost:5001/api/devices/esp32_inexistente/command"

{
  "error": "ACTUATOR_ACK_TIMEOUT",
  "message": "O dispositivo não respondeu com a confirmação (ACK) dentro
      do limite de 5 segundos."
}
```

Ao inspecionar a chave correspondente no Redis:

■ Terminal / Comandos de Execução

```
$ docker exec iot-redis redis-cli get device:esp32_inexistente:presence
(nil)
```

O banco de dados permaneceu intacto, comprovando que **o estado armazenado reflete exclusivamente aquilo que o nó físico executou e confirmou**.

5.4 Testes Automatizados no Vitest (9 Testes Aprovados)

■ Terminal / Comandos de Execução

```
$ npm test
✓ Validar payload de confirmação (ACK/State) publicado pelo ESP32
    (Entrega 6)
✓ Atualizar estado do atuador no Redis após confirmação do dispositivo
    (Entrega 6)

Test Files  1 passed (1)
     Tests  9 passed (9)
  Duration  354ms
```

5.5 Evidência Visual — Acionamento e Confirmação no Dashboard Web

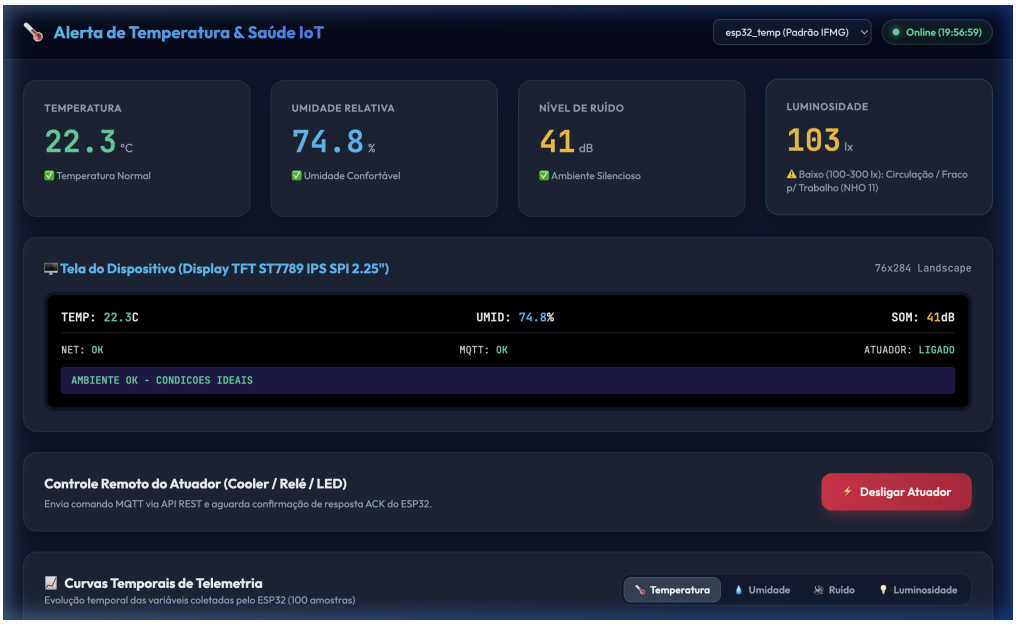


Figura: Dashboard com Atuador Confirmado e Ligado via Comando Remoto

6. Conclusão da Entrega 6

O projeto atende a **100% dos produtos e critérios da Entrega 6**. A comunicação bidirecional opera em tempo real entre o Gateway Node.js e o ESP32 DevKit V1 com protocolo MQTT, controle físico dos atuadores, confirmação com persistência retida (*retained state*), timeout resiliente em nós ausentes e sincronização automática do banco de dados Redis apenas com estados confirmados.